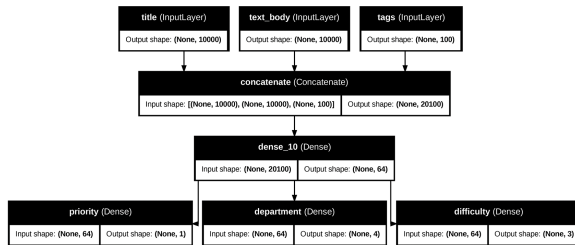


Functional API: Trích xuất Tính năng (Feature Extraction)

- Mô hình chứa các *nút trung gian* (*Intermediate Nodes*). Ta có thể tái sử dụng chúng.
- Trích xuất layer tại mốc số 4 (Dense layer) và bổ sung 1 nhánh difficulty (độ khó).

```
1 features = model.layers[4].output
2 difficulty = layers.Dense(
3     3, activation="softmax", name="difficulty"
4 )(features)
5
6 new_model = keras.Model(
7     inputs=[title, text_body, tags],
8     outputs=[priority, department, difficulty]
9 )
```



Hình 7.5: Mở rộng đồ thị

Phương pháp 3: Model Subclassing

- Phương pháp cấp thấp, linh hoạt nhất.
- Kế thừa trực tiếp Lập trình Hướng đối tượng (OOP) từ lớp `keras.Model`.
- Khai báo các Layer (chứa trọng số) tại hàm tạo `__init__()`.
- Định nghĩa dòng chảy Toán học ngầm (Forward pass) tại hàm `call()`.
- **Ưu điểm:** Cung cấp khả năng kiểm soát tuyệt đối, ví dụ sử dụng vòng lặp `for`, đệ quy bên trong mạng Nơ-ron.

Model Subclassing: Mã nguồn

Viết lại Ticket Classifier dưới dạng Class:

```
1 class CustomerTicketModel(keras.Model):
2     def __init__(self, num_departments):
3         super().__init__() # Always call parent constructor
4         self.concat_layer = layers.Concatenate()
5         self.mixing_layer = layers.Dense(64, activation="relu")
6         self.priority_scorer = layers.Dense(1, activation="sigmoid")
7         self.department_classifier = layers.Dense(num_departments, activation="
softmax")
8
9     def call(self, inputs): # Forward pass logic
10         title = inputs["title"]
11         text_body = inputs["text_body"]
12         tags = inputs["tags"]
13
14         features = self.concat_layer([title, text_body, tags])
15         features = self.mixing_layer(features)
16         priority = self.priority_scorer(features)
17         department = self.department_classifier(features)
18         return priority, department
```

Model Subclassing: Cảnh báo

- **Nhược điểm:** Bạn đang viết "mã byte" dạng hộp đen (Black-box), không còn là đồ thị (Graph) rõ ràng nữa.
- Không thể gọi hàm `summary()` để in ra cấu trúc các nút kết nối.
- Không thể sử dụng `plot_model()` để vẽ ảnh cấu trúc mạng.
- Không thể dùng kĩ thuật **Feature Extraction** do không thể truy xuất `model.layers[i].output`.
- Người lập trình hoàn toàn chịu trách nhiệm cho các lỗi logic bên trong `call()`.

Tùy biến Keras: Khi các hàm có sẵn là không đủ

- API `model.fit()` rất tốt cho hầu hết bài toán thực tế.
- Nhưng với các nhà nghiên cứu, họ thường phát minh ra các hàm Mất mát (Losses) mới, thước đo (Metrics) mới, hoặc những thuật toán Huấn luyện phi tiêu chuẩn (GANs).
- Keras cho phép ghi đè hoàn toàn:
 - Custom Loss
 - Custom Metric
 - Custom Training Step (`train_step()`)
 - Custom Training Loop (Low-level GradientTape)

Tự định nghĩa Hàm Mất mát (Custom Losses)

Đôi khi hàm *Mean Squared Error (MSE)* mặc định là không đủ. Ta có thể viết một hàm (function) đơn giản:

```
1 import tensorflow as tf
2
3 def custom_mean_squared_error(y_true, y_pred):
4     return tf.reduce_mean(tf.square(y_true - y_pred))
5
6 model.compile(optimizer="adam", loss=custom_mean_squared_error)
```

Hoặc kế thừa lớp `keras.losses.Loss`:

```
1 class CustomMSE(keras.losses.Loss):
2     def __init__(self, name="custom_mse", **kwargs):
3         super().__init__(name=name, **kwargs)
4
5     def call(self, y_true, y_pred):
6         return tf.reduce_mean(tf.square(y_true - y_pred))
```

Tự định nghĩa Thước đo (Custom Metrics)

Metrics khác với Loss ở chỗ nó mang **Trạng thái (Stateful)**.

- Loss: Tính cục bộ trên mỗi Batch (Lô).
- Metric: Phải cộng dồn trên nhiều Batch và tính trung bình trong suốt 1 Epoch.
- Kế thừa lớp `keras.metrics.Metric` yêu cầu 3 hàm:
 - 1 `update_state()`: Cộng dồn kết quả đếm cho mỗi Batch.
 - 2 `result()`: Tính toán và trả về kết quả hiển thị cho người dùng.
 - 3 `reset_state()`: Xóa sạch bộ đếm về 0 khi bắt đầu một Epoch mới.

Custom Metrics: Cài đặt thực tế (Root Mean Squared Error)

```
1 class RootMeanSquaredError(keras.metrics.Metric):
2     def __init__(self, name="rmse", **kwargs):
3         super().__init__(name=name, **kwargs)
4         # Declare state variables
5         self.mse_sum = self.add_weight(name="mse_sum", initializer="zeros")
6         self.total_samples = self.add_weight(name="total_samples", initializer="
zeros")
7
8     def update_state(self, y_true, y_pred, sample_weight=None):
9         mse = tf.reduce_mean(tf.square(y_true - y_pred))
10        self.mse_sum.assign_add(mse)
11        self.total_samples.assign_add(1.0)
12
13    def result(self):
14        # Return sqrt of accumulated mean MSE
15        return tf.sqrt(self.mse_sum / self.total_samples)
16
17    def reset_state(self):
18        self.mse_sum.assign(0.)
19        self.total_samples.assign(0.)
```

Viết lại Vòng lặp Huấn luyện với GradientTape

Thay vì gọi `model.fit()`, ta có thể sử dụng `tf.GradientTape` để tự điều khiển phép tính **Đạo hàm riêng (Backpropagation)**:

$$\Delta W = -\eta \cdot \frac{\partial L}{\partial W} \quad (1)$$

Quy trình 4 bước của 1 Batch (Lô dữ liệu):

- ➊ Mở không gian bộ nhớ (Tape), truyền dữ liệu qua Mạng.
- ➋ Tính toán Loss L .
- ➌ Dùng Tape truy xuất Gradient $\frac{\partial L}{\partial W}$ của các Trọng số.
- ➍ Gửi Gradient cho Trình tối ưu hóa (Optimizer) để cập nhật Trọng số W .

Custom Training Loop: Mã nguồn GradientTape

Đây là kiến thức quan trọng nhất dành cho một Chuyên gia Học sâu.

```
1 optimizer = keras.optimizers.Adam(learning_rate=1e-3)
2 loss_fn = keras.losses.SparseCategoricalCrossentropy()
3
4 for epoch in range(3): # Epoch loop
5     for step, (x_batch, y_batch) in enumerate(train_dataset): # Batch loop
6         # 1. Open Tape
7         with tf.GradientTape() as tape:
8             # 2. Feed-forward and calculate loss
9             logits = model(x_batch, training=True)
10            loss_value = loss_fn(y_batch, logits)
11
12            # 3. Compute Gradients
13            gradients = tape.gradient(loss_value, model.trainable_weights)
14
15            # 4. Apply Gradients (Update weights)
16            optimizer.apply(gradients, model.trainable_weights)
```

Ghi đè hàm `train_step()`

Để kết hợp vòng lặp thủ công với tính năng của `fit()` (hiển thị thanh tiến trình), bạn có thể ghi đè `train_step()`:

```
1 class CustomModel(keras.Model):
2     def train_step(self, data):
3         inputs, targets = data
4         with tf.GradientTape() as tape:
5             predictions = self(inputs, training=True)
6             loss = self.compute_loss(y=targets, y_pred=predictions)
7
8             gradients = tape.gradient(loss, self.trainable_weights)
9             self.optimizer.apply_gradients(gradients, self.trainable_weights)
10
11         # Update Metrics
12         for metric in self.metrics:
13             if metric.name == "loss":
14                 metric.update_state(loss)
15             else:
16                 metric.update_state(targets, predictions)
17         return {m.name: m.result() for m in self.metrics}
```

Bạn vẫn có thể dùng `'model.fit()'` và tận hưởng mọi Callback.

Sức mạnh của Callbacks

- Huấn luyện mô hình Deep Learning có thể tốn hàng giờ hoặc nhiều tuần.
- Bạn không thể cứ ngồi nhìn màn hình suốt quá trình đó để can thiệp kịp thời.
- **Callbacks** là các "hook" (chốt chặn) được kích hoạt tự động vào các thời điểm cụ thể (Đầu/cuối Epoch, Đầu/cuối Batch).
- **Ứng dụng:** Tự dừng sớm, Lưu trọng số, Thay đổi tốc độ học...

Các Callbacks tích hợp: EarlyStopping & ModelCheckpoint

EarlyStopping: Tự động dừng huấn luyện khi val_loss không còn giảm (Ngăn Overfitting).

ModelCheckpoint: Tự động sao lưu Trọng số Tốt nhất xuống ổ cứng.

```
1 callbacks_list = [  
2     keras.callbacks.EarlyStopping(  
3         monitor="val_accuracy",  
4         patience=2, # Stop if no improvement after 2 epochs  
5     ),  
6     keras.callbacks.ModelCheckpoint(  
7         filepath="my_checkpoint.keras",  
8         monitor="val_loss",  
9         save_best_only=True, # Only save model with lowest val_loss  
10    )  
11 ]  
12 model.fit(..., callbacks=callbacks_list, ...)
```

Tùy biến Callback (Custom Callback)

Bạn có thể tự tạo Callback bằng cách kế thừa `keras.callbacks.Callback`. Các hàm sự kiện bạn có thể ghi đè:

```
1 class MyCustomCallback(keras.callbacks.Callback):  
2     def on_epoch_begin(self, epoch, logs=None):  
3         print(f"Start epoch {epoch}")  
4  
5     def on_epoch_end(self, epoch, logs=None):  
6         print(f"End epoch {epoch}")  
7  
8     def on_batch_begin(self, batch, logs=None): ...  
9     def on_batch_end(self, batch, logs=None): ...  
10    def on_train_begin(self, logs=None): ...  
11    def on_train_end(self, logs=None): ...
```

Custom Callback: Ví dụ ghi lại Loss theo Batch

```
1 import matplotlib.pyplot as plt
2
3 class LossHistory(keras.callbacks.Callback):
4     def on_train_begin(self, logs=None):
5         self.per_batch_losses = [] # Store loss for every batch
6
7     def on_batch_end(self, batch, logs=None):
8         self.per_batch_losses.append(logs.get("loss"))
9
10    def on_train_end(self, logs=None):
11        plt.clf()
12        plt.plot(range(len(self.per_batch_losses)), self.per_batch_losses, label="
Training loss")
13        plt.xlabel(f"Batches (Total: {len(self.per_batch_losses)})")
14        plt.ylabel("Loss")
15        plt.legend()
16        plt.savefig("loss_history_callback_example.png")
```